

Working with Netflix Ribbon

- ✓ **Netflix Ribbon** is an Inter Process Communication (IPC) cloud library.
- ✓ Ribbon mainly provides **client-side load balancing algorithms**.
- ✓ Along with the client-side load balancing algorithms, Ribbon provides also following features:
 - Service Discovery Integration
 - Fault Tolerance
 - Configurable load-balancing rules
- ✓ **Service Discovery Integration**
 - Ribbon load balancers provide service discovery in dynamic environments like a cloud.
 - Integration with Eureka and Netflix service discovery component is included in the ribbon library
- ✓ **Fault Tolerance**
 - Ribbon API can dynamically determine whether the servers are up and running in a live environment and can detect those servers that are down
- ✓ **Configurable load-balancing rules**
 - Ribbon supports Various **load-balancing** implementations out of the box and also supports defining custom rules
 - Very **load-balancing** implementation is sub type of IRule and some of the implementations of IRule Interface are as follows.
 - RandomRule
 - RoundRobinRule
 - AvailabilityFilteringRule,
 - BestAvailableRule
 - RetryRule
 - WeightedResponseTimeRule



✓ Configurable Ping Styles.

- Ribbon supports Various **Ping mechanisms** out of the box.
- Very **Ping mechanism** is sub type of IPing and some of the implementations of IPing Interface are as follows.
 - DummyPing
 - PingUrl
 - NoOpPing

✓ Ribbon component contains set of following beans.

- **IClientConfig** - which stores client configuration for a client or load balancer,
- **ILoadBalancer** - which represents a software load balancer,
- **ServerList** - which defines how to get a list of servers to choose from,
- **IRule** - which describes a load balancing strategy, and
- **IPing** - which says how periodic pings of a server are performed?

✓ When you specify the **Eureka Client** in the MicroService then **Netflix Ribbon** library will be added to the project automatically.

Working with Spring Cloud Load Balancer

- ✓ Spring Cloud is currently on the verge of large changes.
- ✓ Spring Cloud Team has planned to move all Spring Cloud Netflix components to the maintenance mode starting from Spring Cloud Greenwich Release Train.
- ✓ The maintenance mode only does not include Eureka, which still will be supported
- ✓ On this way, the biggest change is a replacement of Ribbon client into Spring Cloud Load Balancer.
- ✓ **Spring Cloud Load Balancer** component is still under active development, so we could expect some new features in the near future.
- ✓ Netflix Ribbon client is a stable solution, but unfortunately not developed anymore.
- ✓ Spring Cloud Load Balancer provides a simple round robin rule for load balancing between multiple instances of a single service.
- ✓ When you specify the **Eureka Client** in the MicroService then **Spring Cloud Load Balancer** library will be added to the project automatically.



- ✓ When you are using the Spring Cloud **2020.0.x Release aka Ilford** – then **Spring Cloud Load Balancer** is used as a **default** client-side load balancer for REST-based communication between applications.
- ✓ When you are using the Spring Cloud **Greenwich Release** or Spring Cloud **Hoxton Release** – then **Ribbon** is still used as a **default** client-side load balancer for REST-based communication between applications.
- ✓ If you want to use **Cloud Load Balancer** with Spring Cloud **Greenwich Release** or Spring Cloud **Hoxton Release** – then **do the following Two Steps**.

1) Disable Ribbon by writing the following property in application properties.

spring.cloud.loadbalancer.ribbon.enabled=false

2) Also exclude the ribbon dependencies from Eureka Dependency as follows

```
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
  <exclusions>

    <exclusion>
      <groupId>org.springframework.cloud</groupId>
      <artifactId>spring-cloud-starter-ribbon</artifactId>
    </exclusion>

    <exclusion>
      <groupId>org.springframework.cloud</groupId>
      <artifactId>spring-cloud-starter-netflix-ribbon</artifactId>
    </exclusion>

    <exclusion>
      <groupId>com.netflix.ribbon</groupId>
      <artifactId>ribbon-eureka</artifactId>
    </exclusion>

  </exclusions>
</dependency>
```



Invoking One MS from another MS

Following are the Options to invoke One MicroService from another MicroService

A) Without Any Load-Balancers

Opt1-Using-RestTemplate

Opt2-Using-Feign

Opt3-Using-DiscoveryClient

B) With Spring-Cloud Load-Balancer

Opt-4- Eureka-Spring-Cloud-RestTemplate

Opt-5- Eureka-Spring-Cloud-LoadBalancerClient

Opt-6- Eureka-Spring-Cloud-Feign

C) With Ribbon Load-Balancer

Opt-7- Eureka-Ribbon-RestTemplate

Opt-8- Eureka-Ribbon-LoadBalancerClient

Opt-9- Eureka-Ribbon-Feign



Opt1-Using-RestTemplate

Option 1: Opt1-Using-RestTemplate(without Eureka and without Load Balancers)

Working Steps:

- 1) Take the Workspace: **Using-RestTemplate**
- 2) Create the MicroService – BookPriceMS
- 3) Create the MicroService - BookSearchMS
- 4) Start the 3 Instances of BookPriceMS
9000, 9100, 9200
- 5) Start the 1 Instance of BookSearchMS
8000
- 6) Open the Browser and hit the following URL
<http://localhost:8000/mybook/105>

BookPriceMS – Files Required

1. BookPriceApplication.java	2. BookPriceInfo.java
3. BookPriceController.java	4. application.properties
5. pom.xml	

1. BookPriceApplication.java

```
package com.jlcindia.bookprice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@SpringBootApplication
public class BookPriceApplication {
    public static void main(String args[]) {
        SpringApplication.run(BookPriceApplication.class, args);
    }
}
```



2. BookPriceInfo.java

```
package com.jlcindia.bookprice;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
public class BookPriceInfo {
    private Integer bookId;
    private double price;
    private double offer;
    private String serverPort;

    //Constructors
    //Setters and Getters
}
```

3. BookPriceController.java

```
package com.jlcindia.bookprice;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.web.bind.annotation.CrossOrigin;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RestController;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@CrossOrigin
@RestController
public class BookPriceController {

    @Value("${server.port}")
    String serverPort;

    @GetMapping("/bookPrice/{bookId}")
    public BookPriceInfo getBookPrice(@PathVariable Integer bookId) {
        System.out.println("---BookPriceController---getBookPrice()--: "+serverPort);
        BookPriceInfo bookPriceInfo = new BookPriceInfo(bookId, 5000, 50,serverPort);

        return bookPriceInfo;
    }
}
```



4. application.properties

spring.application.name=MyBookPriceMS

server.port=9000

logging.level.root=INFO

logging.pattern.console=%-5level %logger{36} - %msg%n

5. pom.xml

```
<project ...>
<modelVersion>4.0.0</modelVersion>
<groupId>com.jlcindia.bookprice</groupId>
<artifactId>BookPriceMS</artifactId>
<version>1.0-SNAPSHOT</version>
<packaging>jar</packaging>

<name>MyBookPriceMS</name>
<url>https://www.jlcindia.com</url>

<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.5.6</version>
</parent>

<dependencies>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-devtools</artifactId>
</dependency>

</dependencies>

<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
```



```
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>
</project>
```

BookSearchMS – Files Required

1. BookSearchApplication.java	2. BookPriceInfo.java
3. BookInfo.java	4. BookSearchController.java
5. application.properties	6. pom.xml

1. BookSearchApplication.java

```
package com.jlcindia.booksearch;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@SpringBootApplication
public class BookSearchApplication {
    public static void main(String args[]) {
        SpringApplication.run(BookSearchApplication.class, args);
    }
}
```

2. BookPriceInfo.java

```
package com.jlcindia.booksearch;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
public class BookPriceInfo {
    private Integer bookId;
    private double price;
    private double offer;
    private String serverPort;

    //Constructors
    //Setters and Getters
}
```




3. BookInfo.java

```
package com.jlcindia.booksearch;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
public class BookInfo {
    private Integer bookId;
    private String bookName;
    private String author;
    private String publications;
    private String category;
    private double price;
    private double offer;
    private String serverPort;

    //Constructors
    //Setters and Getters
}
```

4. BookSearchController.java

```
package com.jlcindia.booksearch;

import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.CrossOrigin;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.client.RestTemplate;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@CrossOrigin
@RestController
public class BookSearchController {

    @GetMapping("/mybook/{bookId}")
    public BookInfo getBookById(@PathVariable Integer bookId) {
        System.out.println("---BookController---getBookById()-----");

        BookInfo bookInfo=new BookInfo();
    }
}
```



```
bookInfo.setBookId(bookId);
bookInfo.setBookName("Master Spring MicroServices");
bookInfo.setAuthor("Srinivas Dande");
bookInfo.setCategory("Java");
bookInfo.setPublications("JLC");

//Need to Invoke BookPriceMS
// Start Here

RestTemplate restTemp = new RestTemplate();
String baseUrl = "http://localhost:9000"; //Think on this
String apiURL= "/bookPrice/"+bookId;
String endpoint = baseUrl+apiURL;

ResponseEntity<BookPriceInfo> respEntity=
restTemp.getForEntity(endpoint, BookPriceInfo.class);

BookPriceInfo bookPriceInfo = respEntity.getBody();

bookInfo.setPrice(bookPriceInfo.getPrice());
bookInfo.setOffer(bookPriceInfo.getOffer());
bookInfo.setServerPort(bookPriceInfo.getServerPort());

// End Here

return bookInfo;
}
}
```

5. application.properties

```
spring.application.name=MyBookSearchMS

server.port=8000

logging.level.root=INFO
logging.pattern.console=%-5level %logger{36} - %msg%n
```



6. pom.xml

```
<project ...>
<modelVersion>4.0.0</modelVersion>
<groupId>com.jlcindia.booksearch</groupId>
<artifactId>BookSearchMS</artifactId>
<version>1.0-SNAPSHOT</version>
<packaging>jar</packaging>

<name>MyBookSearchMS</name>
<url>https://www.jlcindia.com</url>

<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.5.6</version>
</parent>

<dependencies>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-devtools</artifactId>
</dependency>

</dependencies>

<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>
</project>
```



Opt2-Using-Feign

Option 2: Using Feign (without Eureka and without Load Balancers)

Working Steps:

- 1) Take the Workspace: **Opt2-Using-Feign**
- 2) Copy the following from **Opt-1 Workspace**
BookPriceMS
BookPSearchMS
- 3) DO the Changes required as shown below
- 4) Start the 3 Instances of BookPriceMS
9000, 9100, 9200
- 5) Start the 1 Instance of BookSearchMS
8000
- 6) Open the Browser and hit the following URL
<http://localhost:8000/mybook/105>

BookPriceMS – Files Required

1. BookPriceApplication.java	Same as Opt-1
2. BookPriceInfo.java	Same as Opt-1
3. BookPriceController.java	Same as Opt-1
4. application.properties	Same as Opt-1
5. pom.xml	Same as Opt-1

BookSearchMS – Files Required

1. BookSearchApplication.java	Updated in Opt-2
2. BookPriceInfo.java	Same as Opt-1
3. BookInfo.java	Same as Opt-1
4. BookPriceProxy.java	Newly Added in Opt-2
5. BookSearchController.java	Updated in Opt-2
6. application.properties	Same as Opt-1
7. pom.xml	Updated in Opt-2



1. BookSearchApplication.java

```
package com.jlcindia.booksearch;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.openfeign.EnableFeignClients;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */

@SpringBootApplication
@EnableFeignClients
public class BookSearchApplication {
    public static void main(String args[]) {
        SpringApplication.run(BookSearchApplication.class, args);
    }
}
```

4. BookPriceProxy.java

```
package com.jlcindia.booksearch;

import org.springframework.cloud.openfeign.FeignClient;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */

@FeignClient(name = "MyBookPriceMS", url = "http://localhost:9000")
public interface BookPriceProxy {

    @GetMapping("/bookPrice/{bookId}")
    public BookPriceInfo getBookPrice(@PathVariable Integer bookId);

}
```



5. BookSearchController.java

```
package com.jlcindia.booksearch;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.CrossOrigin;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RestController;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@CrossOrigin
@RestController
public class BookSearchController {

    @Autowired
    BookPriceProxy bookPriceProxy;

    @GetMapping("/mybook/{bookId}")
    public BookInfo getBookById(@PathVariable Integer bookId) {
        System.out.println("---BookController---getBookById()-----");
        BookInfo bookInfo=new BookInfo();

        bookInfo.setBookId(bookId);
        bookInfo.setBookName("Master Spring MicroServices");
        bookInfo.setAuthor("Srinivas Dande");
        bookInfo.setCategory("Java");
        bookInfo.setPublications("JLC");

        //Need to Invoke BookPriceMS
        // Start Here

        BookPriceInfo bookPriceInfo = bookPriceProxy.getBookPrice(bookId);

        bookInfo.setPrice(bookPriceInfo.getPrice());
        bookInfo.setOffer(bookPriceInfo.getOffer());
        bookInfo.setServerPort(bookPriceInfo.getServerPort());

        // End Here
        return bookInfo;
    }
}
```



7. pom.xml

```
<project ...>

<modelVersion>4.0.0</modelVersion>
<groupId>com.jlcindia.booksearch</groupId>
<artifactId>BookSearchMS</artifactId>
<version>1.0-SNAPSHOT</version>
<packaging>jar</packaging>

<name>MyBookSearchMS</name>
<url>https://www.jlcindia.com</url>

<properties>
<java.version>1.8</java.version>
<spring-cloud.version>2020.0.4</spring-cloud.version>
</properties>

<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.5.6</version>
</parent>

<dependencies>

<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-openfeign</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-devtools</artifactId>
</dependency>

</dependencies>
```



```
<dependencyManagement>
<dependencies>
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-dependencies</artifactId>
<version>${spring-cloud.version}</version>
<type>pom</type>
<scope>import</scope>
</dependency>
</dependencies>
</dependencyManagement>

<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>
</project>
```

Option 3: Using-DiscoveryClient

Option 3: Using Eureka and DiscoveryClient (Without Ribbon)

Working Steps:

- 1) Take the Workspace: **Opt3-Using-DiscoveryClient**
- 2) Copy the following from **Opt2 Workspace**
BookPriceMS
BookPSearchMS
- 3) Create Eureka Server Project in this WorkSpace.
- 4) DO the Changes required as shown below
- 5) Start Eureka Server.
- 6) Start the 3 Instances of BookPriceMS
9000, 9100, 9200
- 7) Start the 1 Instance of BookSearchMS
8000
- 8) Open the Browser and hit the following URL
<http://localhost:8000/mybook/105>



Eureka Server: Files required

1. MyEurekaServerApplication.java	2. application.properties
3. pom.xml	

1. MyEurekaServerApplication.java

```
package com.jlcindia.myeurekaserver;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.netflix.eureka.server.EnableEurekaServer;

@SpringBootApplication
@EnableEurekaServer
public class MyEurekaServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(StartMyEurekaServer.class, args);
    }
}
```

2. application.properties

```
spring.application.name=MyEurekaServer
server.port=8761

#telling the server not to register himself in the service registry
eureka.client.register-with-eureka=false
eureka.client.fetchRegistry= false
```

3. pom.xml

```
<project ...>
<modelVersion>4.0.0</modelVersion>
<groupId>com.jlcindia.myeurekaserver</groupId>
<artifactId>MyEurekaServer</artifactId>
<version>1.0</version>
<name>MyEurekaServer</name>
<description> Eureka - Service Registry and Discovery Server</description>

<properties>
<java.version>1.8</java.version>
<spring-cloud.version>2020.0.4</spring-cloud.version>
</properties>
```



```
<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.5.6</version>
</parent>

<dependencies>
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-netflix-eureka-server</artifactId>
</dependency>
</dependencies>

<dependencyManagement>

<dependencies>
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-dependencies</artifactId>
<version>${spring-cloud.version}</version>
<type>pom</type>
<scope>import</scope>
</dependency>
</dependencies>

</dependencyManagement>
<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>
</project>
```



BookPriceMS – Files Required

1. BookPriceApplication.java	Updated in Opt-3
2. BookPriceInfo.java	Same as Opt-2
3. BookPriceController.java	Same as Opt-2
4. application.properties	Updated in Opt-3
5. pom.xml	Updated in Opt-3

1. BookPriceApplication.java

```
package com.jlcindia.bookprice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.netflix.eureka.EnableEurekaClient;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@SpringBootApplication
@EnableEurekaClient
public class BookPriceApplication {
    public static void main(String args[]) {
        SpringApplication.run(BookPriceApplication.class, args);
    }
}
```

4. application.properties

```
spring.application.name=MyBookPriceMS

server.port=9000

eureka.client.service-url.defaultZone=http://localhost:8761/eureka
eureka.client.healthcheck.enabled=true

logging.level.root=INFO
logging.pattern.console=%-5level %logger{36} - %msg%n

management.endpoints.web.exposure.include=*
management.endpoint.health.show-details=always
```



INFO ENDPOINT CONFIGURATION

```
info.app.name=@project.name@
info.app.description=@project.description@
info.app.version=@project.version@
info.app.encoding=@project.build.sourceEncoding@
info.app.java.version=@java.version@
info.app.developer=Srinivas Dande
info.app.company=Java Learning Center
```

5. pom.xml

```
<project ...>

<modelVersion>4.0.0</modelVersion>
<groupId>com.jlcindia.bookprice</groupId>
<artifactId>BookPriceMS</artifactId>
<version>1.0-SNAPSHOT</version>
<packaging>jar</packaging>

<name>MyBookPriceMS</name>
<url>https://www.jlcindia.com</url>

<properties>
<java.version>1.8</java.version>
<spring-cloud.version>2020.0.4</spring-cloud.version>
</properties>

<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.5.6</version>
</parent>

<dependencies>

<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```



```
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-devtools</artifactId>
</dependency>

</dependencies>

<dependencyManagement>
<dependencies>
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-dependencies</artifactId>
<version>${spring-cloud.version}</version>
<type>pom</type>
<scope>import</scope>
</dependency>
</dependencies>
</dependencyManagement>

<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>

</project>
```



BookSearchMS – Files Required

1. BookSearchApplication.java	Updated in Opt-3
2. BookPriceInfo.java	Same as Opt-1
3. BookInfo.java	Same as Opt-1
4. BookSearchController.java	Updated in Opt-3
5. application.properties	Updated in Opt-3
6. pom.xml	Updated in Opt-3

1. BookSearchApplication.java

```
package com.jlcindia.booksearch;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.netflix.eureka.EnableEurekaClient;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@SpringBootApplication
@EnableEurekaClient
public class BookSearchApplication {
    public static void main(String args[]) {
        SpringApplication.run(BookSearchApplication.class, args);
    }
}
```

4. BookSearchController.java

```
package com.jlcindia.booksearch;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.cloud.client.ServiceInstance;
import org.springframework.cloud.client.discovery.DiscoveryClient;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.CrossOrigin;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.client.RestTemplate;
```



```
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@CrossOrigin
@RestController
public class BookSearchController {

    @Autowired
    DiscoveryClient discoveryClient;

    @GetMapping("/mybook/{bookId}")
    public BookInfo getBookById(@PathVariable Integer bookId) {
        System.out.println("---BookController---getBookById()-----");
        BookInfo bookInfo=new BookInfo();

        bookInfo.setBookId(bookId);
        bookInfo.setBookName("Master Spring MicroServices");
        bookInfo.setAuthor("Srinivas Dande");
        bookInfo.setCategory("Java");
        bookInfo.setPublications("JLC");

        //Need to Invoke BookPriceMS
        // Start Here

        List<ServiceInstance> instancesList=
        discoveryClient.getInstances("MyBookPriceMS");

        for(ServiceInstance myInstance:instancesList) {
            System.out.println("Hello : "+myInstance.getUri());
        }

        String baseUrl= instancesList.get(0).getUri().toString(); //Think On this
        System.out.println("Base URL : "+ baseUrl);

        RestTemplate restTemp = new RestTemplate();
        String apiURL= "/bookPrice/"+bookId;
        String endpoint = baseUrl+apiURL;

        ResponseEntity<BookPriceInfo> respEntity=
        restTemp.getForEntity(endpoint, BookPriceInfo.class);

        BookPriceInfo bookPriceInfo = respEntity.getBody();
    }
}
```



```
bookInfo.setPrice(bookPriceInfo.getPrice());
bookInfo.setOffer(bookPriceInfo.getOffer());
bookInfo.setServerPort(bookPriceInfo.getServerPort());

// End Here

return bookInfo;
}
}
```

5. application.properties

spring.application.name=MyBookSearchMS

server.port=8000

eureka.client.service-url.defaultZone=http://localhost:8761/eureka
eureka.client.healthcheck.enabled=true

logging.level.root=INFO
logging.pattern.console=%-5level %logger{36} - %msg%n

management.endpoints.web.exposure.include=*
management.endpoint.health.show-details=always

INFO ENDPOINT CONFIGURATION
info.app.name=@project.name@
info.app.description=@project.description@
info.app.version=@project.version@
info.app.encoding=@project.build.sourceEncoding@
info.app.java.version=@java.version@
info.app.developer=Srinivas Dande
info.app.company=Java Learning Center



6. pom.xml

```
<project ...>
<modelVersion>4.0.0</modelVersion>
<groupId>com.jlcindia.booksearch</groupId>
<artifactId>BookSearchMS</artifactId>
<version>1.0-SNAPSHOT</version>
<packaging>jar</packaging>

<name>MyBookSearchMS</name>
<url>https://www.jlcindia.com</url>

<properties>
<java.version>1.8</java.version>
<spring-cloud.version>2020.0.4</spring-cloud.version>
</properties>

<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.5.6</version>
</parent>

<dependencies>
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-actuator</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-devtools</artifactId>
</dependency>
</dependencies>
```



```
<dependencyManagement>
<dependencies>
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-dependencies</artifactId>
<version>${spring-cloud.version}</version>
<type>pom</type>
<scope>import</scope>
</dependency>
</dependencies>
</dependencyManagement>
<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>
</project>
```

Option 4: -Using-Load Balancer-Backed-RestTemplate

Option 4: Using-Load Balancer-Backed-RestTemplate

Working Steps:

- 1) Take the Workspace: **Opt4-Using-LB-Backed-RT**
- 2) Copy the following from **Opt3 Workspace**
MyEurekaServer
BookPriceMS
BookSearchMS
- 3) DO the Changes required as shown below
- 4) Start Eureka Server.
- 5) Start the 3 Instances of BookPriceMS
9000, 9100, 9200
- 6) Start the 1 Instance of BookSearchMS
8000
- 7) Open the Browser and hit the following URL
<http://localhost:8000/mybook/105>



MyEurekaServer – Files Required

1. MyEurekaServerApplication.java	Same as Opt-3
2. application.properties	Same as Opt-3
3. pom.xml	Same as Opt-3

BookPriceMS – Files Required

1. BookPriceApplication.java	Same as Opt-3
2. BookPriceInfo.java	Same as Opt-3
3. BookPriceController.java	Same as Opt-3
4. application.properties	Same as Opt-3
5. pom.xml	Same as Opt-3

BookSearchMS – Files Required

1. BookSearchApplication.java	Updated in Opt-4
2. BookPriceInfo.java	Same as Opt-3
3. BookInfo.java	Same as Opt-3
4. BookSearchController.java	Updated in Opt-4
5. application.properties	Same as Opt-3
6. pom.xml	Same as Opt-3

1. BookSearchApplication.java

```
package com.jlcindia.booksearch;

import org.springframework.boot.SpringApplication;

import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.client.loadbalancer.LoadBalanced;
import org.springframework.cloud.netflix.eureka.EnableEurekaClient;
import org.springframework.context.annotation.Bean;
import org.springframework.web.client.RestTemplate;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@SpringBootApplication
@EnableEurekaClient
public class BookSearchApplication {
    public static void main(String args[]) {
        SpringApplication.run(BookSearchApplication.class, args);
    }
}
```



```
}  
  
@LoadBalanced  
@Bean  
public RestTemplate getRestTemp() {  
return new RestTemplate();  
}  
  
}
```

4. BookSearchController.java

```
package com.jlcindia.booksearch;  
  
import org.springframework.beans.factory.annotation.Autowired;  
import org.springframework.http.ResponseEntity;  
import org.springframework.web.bind.annotation.CrossOrigin;  
import org.springframework.web.bind.annotation.GetMapping;  
import org.springframework.web.bind.annotation.PathVariable;  
import org.springframework.web.bind.annotation.RestController;  
import org.springframework.web.client.RestTemplate;  
/*  
 * @Author : Srinivas Dande  
 * @company : Java Learning Center  
 */  
  
@CrossOrigin  
@RestController  
public class BookSearchController {  
  
    @Autowired  
    RestTemplate restTemp;  
  
    @GetMapping("/mybook/{bookId}")  
    public BookInfo getBookById(@PathVariable Integer bookId) {  
        System.out.println("---BookController---getBookById()-----");  
        BookInfo bookInfo = new BookInfo();  
  
        bookInfo.setBookId(bookId);  
        bookInfo.setBookName("Master Spring MicroServices");  
        bookInfo.setAuthor("Srinivas Dande");  
        bookInfo.setCategory("Java");  
        bookInfo.setPublications("JLC");
```



```
// Need to Invoke BookPriceMS
// Start Here

String baseUrl = "http://MyBookPriceMS";
String apiURL = "/bookPrice/" + bookId;
String endpoint = baseUrl + apiURL;

ResponseEntity<BookPriceInfo> respEntity = restTemplate.getForEntity(endpoint,
BookPriceInfo.class);

BookPriceInfo bookPriceInfo = respEntity.getBody();

bookInfo.setPrice(bookPriceInfo.getPrice());
bookInfo.setOffer(bookPriceInfo.getOffer());
bookInfo.setServerPort(bookPriceInfo.getServerPort());

// End Here

return bookInfo;
}
}
```

Option 5: Using LoadBalancerClient

Option 5: Using LoadBalancerClient

Working Steps:

- 1) Take the Workspace: **Opt5-Using-LBClient**
- 2) Copy the following from **Opt4 Workspace**
MyEurekaServer
BookPriceMS
BookSearchMS
- 3) DO the Changes required as shown below
- 4) Start Eureka Server.
- 5) Start the 3 Instances of BookPriceMS
9000, 9100, 9200
- 6) Start the 1 Instance of BookSearchMS
8000
- 7) Open the Browser and hit the following URL
<http://localhost:8000/mybook/105>



MyEurekaServer – Files Required

1. MyEurekaServerApplication.java	Same as Opt-4
2. application.properties	Same as Opt-4
3. pom.xml	Same as Opt-4

BookPriceMS – Files Required

1. BookPriceApplication.java	Same as Opt-4
2. BookPriceInfo.java	Same as Opt-4
3. BookPriceController.java	Same as Opt-4
4. application.properties	Same as Opt-4
5. pom.xml	Same as Opt-4

BookSearchMS – Files Required

1. BookSearchApplication.java	Updated in Opt-5
2. BookPriceInfo.java	Same as Opt-4
3. BookInfo.java	Same as Opt-4
4. BookSearchController.java	Updated in Opt-5
5. application.properties	Same as Opt-4
6. pom.xml	Same as Opt-4

1. BookSearchApplication.java

```
package com.jlcindia.booksearch;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.netflix.eureka.EnableEurekaClient;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@SpringBootApplication
@EnableEurekaClient
public class BookSearchApplication {
    public static void main(String args[]) {
        SpringApplication.run(BookSearchApplication.class, args);
    }
}
```



4. BookSearchController.java

```
package com.jlcindia.booksearch;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.cloud.client.ServiceInstance;
import org.springframework.cloud.client.loadbalancer.LoadBalancerClient;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.CrossOrigin;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.client.RestTemplate;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */

@CrossOrigin
@RestController
public class BookSearchController {

    @Autowired
    LoadBalancerClient loadBalancerClient;

    @GetMapping("/mybook/{bookId}")
    public BookInfo getBookById(@PathVariable Integer bookId) {
        System.out.println("---BookController---getBookById()-----");
        BookInfo bookInfo = new BookInfo();

        bookInfo.setBookId(bookId);
        bookInfo.setBookName("Master Spring MicroServices");
        bookInfo.setAuthor("Srinivas Dande");
        bookInfo.setCategory("Java");
        bookInfo.setPublications("JLC");

        // Need to Invoke BookPriceMS
        // Start Here

        ServiceInstance instance = loadBalancerClient.choose("MyBookPriceMS");
        String baseUrl=instance.getUri().toString();
        System.out.println("Base URL : "+ baseUrl);
        String apiURL = "/bookPrice/" + bookId;
        String endpoint = baseUrl + apiURL;
```




```
RestTemplate restTemplate=new RestTemplate();  
ResponseEntity<BookPriceInfo> respEntity = restTemplate.getForEntity(endpoint,  
BookPriceInfo.class);
```

```
BookPriceInfo bookPriceInfo = respEntity.getBody();
```

```
bookInfo.setPrice(bookPriceInfo.getPrice());  
bookInfo.setOffer(bookPriceInfo.getOffer());  
bookInfo.setServerPort(bookPriceInfo.getServerPort());
```

```
// End Here
```

```
return bookInfo;  
}  
}
```

Option 6: Using Eureka , Feign

Option 6: Using Eureka , Feign

Working Steps:

- 1) Take the Workspace: **Opt6-Using-Feign-Eureka**
- 2) Copy the following from **Opt5 Workspace**

MyEurekaServer
BookPriceMS
BookSearchMS

- 3) DO the Changes required as shown below
- 4) Start Eureka Server.
- 5) Start the 3 Instances of BookPriceMS
9000, 9100, 9200
- 6) Start the 1 Instance of BookSearchMS
8000
- 7) Open the Browser and hit the following URL
<http://localhost:8000/mybook/105>

MyEurekaServer – Files Required

1. MyEurekaServerApplication.java	Same as Opt-5
2. application.properties	Same as Opt-5
3. pom.xml	Same as Opt-5



BookPriceMS – Files Required

1. BookPriceApplication.java	Same as Opt-5
2. BookPriceInfo.java	Same as Opt-5
3. BookPriceController.java	Same as Opt-5
4. application.properties	Same as Opt-5
5. pom.xml	Same as Opt-5

BookSearchMS – Files Required

1. BookSearchApplication.java	Updated in Opt-6
2. BookPriceInfo.java	Same as Opt-5
3. BookInfo.java	Same as Opt-5
4. BookPriceProxy.java	Newly Added in Opt-6
5. BookSearchController.java	Updated in Opt-6
6. application.properties	Same as Opt-5
7. pom.xml	Updated in Opt-6

1. BookSearchApplication.java

```
package com.jlcindia.booksearch;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.netflix.eureka.EnableEurekaClient;
import org.springframework.cloud.openfeign.EnableFeignClients;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */

@SpringBootApplication
@EnableEurekaClient
@EnableFeignClients
public class BookSearchApplication {
    public static void main(String args[]) {
        SpringApplication.run(BookSearchApplication.class, args);
    }
}
```



4. BookPriceProxy.java

```
package com.jlcindia.booksearch;

import org.springframework.cloud.openfeign.FeignClient;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@FeignClient( "MyBookPriceMS")
public interface BookPriceProxy {

    @GetMapping("/bookPrice/{bookId}")
    public BookPriceInfo getBookPrice(@PathVariable Integer bookId);

}
```

5. BookSearchController.java

```
package com.jlcindia.booksearch;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.CrossOrigin;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RestController;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */

@CrossOrigin
@RestController
public class BookSearchController {

    @Autowired
    BookPriceProxy bookPriceProxy;

    @GetMapping("/mybook/{bookId}")
    public BookInfo getBookById(@PathVariable Integer bookId) {
        System.out.println("---BookController---getBookById()-----");
        BookInfo bookInfo = new BookInfo();
    }
}
```



```
bookInfo.setBookId(bookId);
bookInfo.setBookName("Master Spring MicroServices");
bookInfo.setAuthor("Srinivas Dande");
bookInfo.setCategory("Java");
bookInfo.setPublications("JLC");

// Need to Invoke BookPriceMS
// Start Here

BookPriceInfo bookPriceInfo = bookPriceProxy.getBookPrice(bookId);

bookInfo.setPrice(bookPriceInfo.getPrice());
bookInfo.setOffer(bookPriceInfo.getOffer());
bookInfo.setServerPort(bookPriceInfo.getServerPort());
// End Here

return bookInfo;
}
}
```

7. pom.xml

```
<project ...>
<modelVersion>4.0.0</modelVersion>
<groupId>com.jlcindia.booksearch</groupId>
<artifactId>BookSearchMS</artifactId>
<version>1.0-SNAPSHOT</version>
<packaging>jar</packaging>
<name>MyBookSearchMS</name>
<url>https://www.jlcindia.com</url>

<properties>
<java.version>1.8</java.version>
<spring-cloud.version>2020.0.4</spring-cloud.version>
</properties>

<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.5.6</version>
</parent>

<dependencies>
<dependency>
<groupId>org.springframework.cloud</groupId>
```



```
<artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-openfeign</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-actuator</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-devtools</artifactId>
</dependency>
</dependencies>

<dependencyManagement>
<dependencies>
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-dependencies</artifactId>
<version>${spring-cloud.version}</version>
<type>pom</type>
<scope>import</scope>
</dependency>
</dependencies>
</dependencyManagement>
<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>
</project>
```



Option 7: Using Ribbon Load Balancer-backed RestTemplate

Option 7: Using Ribbon Load Balancer-backed RestTemplate

Working Steps:

- 1) Take the Workspace: **Opt7-Using-LB-Backed-RT-Ribbon**
- 2) Copy the following from **Opt4 Workspace**
MyEurekaServer
BookPriceMS
BookSearchMS
- 3) DO the Changes required as shown below
- 4) Start Eureka Server.
- 5) Start the 3 Instances of BookPriceMS - **9000, 9100, 9200**
- 6) Start the 1 Instance of BookSearchMS - **8000**
- 7) Open the Browser and hit the following URL
<http://localhost:8000/mybook/105>

MyEurekaServer – Files Required

1. MyEurekaServerApplication.java	Same as Opt-4
2. application.properties	Same as Opt-4
3. pom.xml	Updated in Opt-7

BookPriceMS – Files Required

1. BookPriceApplication.java	Same as Opt-4
2. BookPriceInfo.java	Same as Opt-4
3. BookPriceController.java	Same as Opt-4
4. application.properties	Same as Opt-4
5. pom.xml	Updated in Opt-7

BookSearchMS – Files Required

1. BookSearchApplication.java	Same as Opt-4
2. BookPriceInfo.java	Same as Opt-4
3. BookInfo.java	Same as Opt-4
4. BookSearchController.java	Same as Opt-4
5. application.properties	Same as Opt-4
6. pom.xml	Updated in Opt-7



- In all 3 Apps, Only pom.xml has to be updated.
- Just the Cloud Version and Boot Version as Follows.

```
<properties>
    <java.version>1.8</java.version>
    <spring-cloud.version>Hoxton.SR8</spring-cloud.version>
</properties>
```

```
<parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>2.3.3.RELEASE</version>
</parent>
```

- When you use Hoxton Release of Spring Cloud then Ribbon Load Balancer will be used as default.

Option 8: Using LoadBalancerClient with Ribbon

Option 8: Using LoadBalancerClient with Ribbon

Working Steps:

- 1) Take the Workspace: **Opt8-Using-LBClient-Ribbon**
- 2) Copy the following from **Opt5 Workspace**
MyEurekaServer
BookPriceMS
BookSearchMS
- 3) DO the Changes required as shown below
- 4) Start Eureka Server.
- 5) Start the 3 Instances of BookPriceMS - **9000, 9100, 9200**
- 6) Start the 1 Instance of BookSearchMS - **8000**
- 7) Open the Browser and hit the following URL
<http://localhost:8000/mybook/105>

MyEurekaServer – Files Required

1. MyEurekaServerApplication.java	Same as Opt-5
2. application.properties	Same as Opt-5
3. pom.xml	Updated in Opt-8

BookPriceMS – Files Required

1. BookPriceApplication.java	Same as Opt-5
2. BookPriceInfo.java	Same as Opt-5
3. BookPriceController.java	Same as Opt-5
4. application.properties	Same as Opt-5
5. pom.xml	Updated in Opt-8

BookSearchMS – Files Required

1. BookSearchApplication.java	Same as Opt-5
2. BookPriceInfo.java	Same as Opt-5
3. BookInfo.java	Same as Opt-5
4. BookSearchController.java	Same as Opt-5
5. application.properties	Same as Opt-5
6. pom.xml	Updated in Opt-8

- In all 3 Apps, Only pom.xml has to be updated.
- Just the Cloud Version and Boot Version as Follows.

```
<properties>
    <java.version>1.8</java.version>
    <spring-cloud.version>Hoxton.SR8</spring-cloud.version>
</properties>
```

```
<parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>2.3.3.RELEASE</version>
</parent>
```

- When you use Hoxton Release of Spring Cloud then Ribbon Load Balancer will be used as default.



Option 9: Using Eureka , Feign and Ribbon

Option 9: Using Eureka , Feign and Ribbon

Working Steps:

- 1) Take the Workspace: **Opt9-Using-Feign-Eureka-Ribbon**
- 2) Copy the following from **Opt6 Workspace**
MyEurekaServer
BookPriceMS
BookSearchMS
- 3) DO the Changes required as shown below
- 4) Start Eureka Server.
- 5) Start the 3 Instances of BookPriceMS
9000, 9100, 9200
- 6) Start the 1 Instance of BookSearchMS
8000
- 7) Open the Browser and hit the following URL
<http://localhost:8000/mybook/105>

MyEurekaServer – Files Required

1. MyEurekaServerApplication.java	Same as Opt-6
2. application.properties	Same as Opt-6
3. pom.xml	Updated in Opt-9

BookPriceMS – Files Required

1. BookPriceApplication.java	Same as Opt-6
2. BookPriceInfo.java	Same as Opt-6
3. BookPriceController.java	Same as Opt-6
4. application.properties	Same as Opt-6
5. pom.xml	Updated in Opt-9



BookSearchMS – Files Required

1. BookSearchApplication.java	Same as Opt-6
2. BookPriceInfo.java	Same as Opt-6
3. BookInfo.java	Same as Opt-6
4. BookPriceProxy.java	Same as Opt-6
5. BookSearchController.java	Same as Opt-6
6. application.properties	Same as Opt-6
7. pom.xml	Updated in Opt-9

- In all 3 Apps, Only pom.xml has to be updated.
- Just the Cloud Version and Boot Version as Follows.

```
<properties>  
    <java.version>1.8</java.version>  
    <spring-cloud.version>Hoxton.SR8</spring-cloud.version>  
</properties>
```

```
<parent>  
    <groupId>org.springframework.boot</groupId>  
    <artifactId>spring-boot-starter-parent</artifactId>  
    <version>2.3.3.RELEASE</version>  
</parent>
```

- When you use Hoxton Release of Spring Cloud then Ribbon Load Balancer will be used as default.